

Angular a Nivel Senior: Guía Práctica y de Referencia

Apuntes de trabajo

19 de octubre de 2025

Índice

1	Introducción	2
2	Fundamentos prácticos: de cero a tu primer componente	2
2.1	Antes de Angular: cómo funciona la web	2
2.2	Qué es un componente	2
2.3	Data binding: conecta datos con la vista	2
2.4	Control de flujo en la plantilla	3
2.5	Servicios e inyección de dependencias (DI)	3
2.6	HTTP básico	3
2.7	Routing básico	4
3	Ecosistema Angular actual	4
4	TypeScript para Angular (rápido y efectivo)	5
5	Arquitectura y organización	5
5.1	Standalone y bootstrap	5
5.2	Inyección de dependencias avanzada	6
6	Componentes profesionales	6
6.1	Inputs/Outputs, Signals y Change Detection	6
6.2	Proyección, referencias y hooks	7
7	Formularios tipados y validaciones	8
7.1	Reactive Forms	8
8	Routing funcional, lazy y resolución de datos	9
9	HTTP profesional: interceptores, errores, caché	9
10	Estado: RxJS, Signals y NgRx	10
10.1	RxJS en componentes	10
10.2	Signals para estado local	10
10.3	NgRx moderno	11
11	Rendimiento y escalabilidad	11
12	SSR, Hydration y despliegue	12
13	Testing eficaz	12
14	Accesibilidad, i18n y seguridad	12
15	Herramientas y calidad	13
16	Patrones y anti-patrones	13
17	Checklist de revisión senior	13

18 Ruta de aprendizaje a experto (temario por niveles)	13
18.1 1. Fundamentos del ecosistema Angular	13
18.2 2. Estructura básica de un proyecto	14
18.3 3. Templates y data binding	14
18.4 4. Comunicación entre componentes	14
18.5 5. Servicios e inyección de dependencias	14
18.6 6. Ruteo y navegación	14
18.7 7. Formularios	14
18.8 8. Observables y RxJS	14
18.9 9. HTTP Client	14
18.10 10. Arquitectura avanzada	14
18.11 11. Gestión de estado	15
18.12 12. Rendimiento y optimización	15
18.13 13. Testing	15
18.14 14. Despliegue y DevOps	15
18.15 15. Ecosistema avanzado	15
18.16 16. Buenas prácticas	15
19 TypeScript avanzado y tipado estricto en Angular	15
19.1 Tipos condicionales, utilitarios y “type-level programming”	15
19.2 Decoradores y metadatos	16
19.3 Chequeo de tipos en plantillas	16
20 Angular por debajo del capó (internals esenciales)	16
20.1 Change Detection: de Zone.js a Signals	16
20.2 Inyección de dependencias en Ivy	16
20.3 Compilación de plantillas (Ivy Instructions)	16
20.4 Hydration y SSR	17
20.5 Builder y empaquetado	17
21 Plan de estudio recomendado (6 semanas)	17
22 Material, CDK y diseño de UI	17
22.1 Angular Material (standalone)	17
22.2 CDK Overlay y Portal	18
23 Animaciones	18
24 PWA y Service Worker	18
25 i18n y localización	19
26 WebSockets y tiempo real	19
27 Web Workers	19
28 Seguridad en el navegador	19
29 Estrategias de precarga y performance de rutas	19
30 Monorepo y DX	20

1 Introducción

Estos apuntes condensan prácticas, patrones y técnicas de Angular modernas (v16+), con foco en componentes standalone, reactividad con Signals, control de flujo de plantillas (@if, @for), vistas diferidas (@defer), DI con inject(), routing funcional y optimización de rendimiento (OnPush, zoneless, SSR + hydration). Se asume solidez en TypeScript.

Consejo Senior

Apunta a: simplicidad, composición, tipado estricto y límites claros entre UI, estado y efectos secundarios.

2 Fundamentos prácticos: de cero a tu primer componente

Esta sección explica Angular desde cero con ejemplos breves y lenguaje simple. Si ya tienes experiencia, salta a las siguientes secciones.

2.1 Antes de Angular: cómo funciona la web

- HTML estructura el contenido (<h1>, <p>, <button>).
- CSS da estilos (colores, tamaños, layouts).
- JavaScript añade comportamiento (responder a clicks, pedir datos al servidor).

Angular es un framework que organiza todo esto para construir aplicaciones grandes de forma mantenible.

2.2 Qué es un componente

Un componente es una pieza de UI con: (1) plantilla (HTML), (2) lógica (TypeScript) y (3) estilos. Reutilizas componentes como *bloques Lego*.

Listing 1: Componente mínimo standalone

```
1 import { Component } from '@angular/core';
2
3 @Component({
4   selector: 'app-hello',           // etiqueta HTML que usarás
5   standalone: true,                // no necesita NgModule
6   template: `<h1>Hola Angular</h1>`
7 })
8 export class HelloComponent {}
```

2.3 Data binding: conecta datos con la vista

- Interpolación: muestra valores con {{ valor }}.
- Property binding: [prop] pasa datos a atributos/propiedades.
- Event binding: (evento) escucha acciones del usuario.

Listing 2: Estado local sencillo con Signals

```
1 import { Component, signal } from '@angular/core';
2
3 @Component({
4   selector: 'app-counter',
5   standalone: true,
```

```

6   template: `
7     <h2>Contador: {{ count() }}</h2>
8     <button (click)="inc()">+1</button>
9     <button (click)="reset()" [disabled]="count() === 0">Reset</button>
10  `
11 })
12 export class CounterComponent {
13   count = signal(0); // estado local
14   inc() { this.count.update(c => c + 1); }
15   reset() { this.count.set(0); }
16 }

```

2.4 Control de flujo en la plantilla

Muestra/oculta y repite elementos sin lógica compleja en TS.

Listing 3: @if y @for con trackBy básico

```

1 <h3>Tareas</h3>
2 @if (todos().length === 0) {
3   <p>No hay tareas</p>
4 } @else {
5   <ul>
6     @for (t of todos(); track t.id) {
7       <li>{{ t.title }}</li>
8     }
9   </ul>
10 }

```

2.5 Servicios e inyección de dependencias (DI)

Un servicio encapsula lógica reutilizable (por ejemplo, hablar con una API). Angular te lo “inyecta” donde lo necesites.

Listing 4: Servicio sencillo e inyección con inject()

```

1 import { Injectable, inject, Component } from '@angular/core';
2
3 @Injectable({ providedIn: 'root' })
4 export class GreeterService {
5   greet(name: string) { return `Hola, ${name}!`; }
6 }
7
8 @Component({ selector: 'app-greeter', standalone: true, template: `<p>{{ msg }}</p>` })
9 export class GreeterComponent {
10   private greeter = inject(GreeterService);
11   msg = this.greeter.greet('Mundo');
12 }

```

2.6 HTTP básico

Listing 5: Pedir datos con HttpClient (GET)

```

1 import { HttpClient } from '@angular/common/http';
2 import { inject, Component, signal } from '@angular/core';
3
4 @Component({ selector: 'app-users', standalone: true, template: `

```

```

5   <button (click)="load()">Cargar</button>
6   <ul>@for (u of users(); track u.id) { <li>{{ u.name }}</li> }</ul>
7   ` })
8   export class UsersComponent {
9     private http = inject(HttpClient);
10    users = signal<ReadOnlyArray<{id:number; name:string}>>([]);
11    load() {
12      this.http.get<{id:number; name:string}[]>('https://jsonplaceholder.typicode
13        .com/users')
14        .subscribe(xs => this.users.set(xs));
15    }
16  }

```

2.7 Routing básico

Define rutas (URL → componente) y navega entre pantallas.

Listing 6: Rutas mínimas con loadComponent

```

1  import { Routes } from '@angular/router';
2
3  export const routes: Routes = [
4    { path: '', loadComponent: () => import('./home.component').then(m => m.
      HomeComponent) },
5    { path: 'about', loadComponent: () => import('./about.component').then(m => m
      .AboutComponent) },
6    { path: '**', redirectTo: '' }
7  ];

```

Consejo Senior

Sigue este orden para aprender: componentes → data binding → servicios/DI → HTTP → routing. Cuando lo domines, vuelve a la sección de **Signals**, **@defer** y **OnPush** para optimizar.

3 Glosario ultra básico (palabras difíciles, explicado simple)

- **Arranque (bootstrap):** “encender” la app. En Angular se hace en `main.ts` con `bootstrapApplication(AppComponent, providers: [...])`. No tiene relación con **Bootstrap** (framework de CSS).
- **Componente:** pieza de la pantalla con su HTML (plantilla) y lógica (TypeScript). Piensa en bloques de Lego.
- **Plantilla (template):** el HTML que renderiza el componente. Puede contener expresiones como `{{ titulo }}` y control de flujo `@if/@for`.
- **Estado:** datos que cambian con el tiempo (contador, lista). Con **Signals** se guarda con `signal()` y se deriva con `computed()`.
- **Servicio (service):** clase para lógica reutilizable (hablar con API, cache). No dibuja UI.
- **Inyección de dependencias (DI):** Angular te “entrega” servicios donde los pides con `inject(MiServicio)`.
- **Provider:** declaración que dice “para este token, usa esta implementación/valor”. Sirve para registrar servicios/configuraciones.
- **Ruta (route)/Router:** relación URL → componente. Permite navegar entre pantallas sin recargar la página.
- **HttpClient:** utilidad para hacer GET/POST/... a APIs.

- **Interceptor:** pieza que envuelve cada petición HTTP (añadir token, reintentos, cache).
- **Guard:** función que decide si se puede entrar a una ruta (por ejemplo, si hay sesión).
- **Resolver:** trae datos antes de cargar una ruta.
- **Standalone:** componentes/servicios que no necesitan NgModule. Es el estilo moderno por defecto.
- **@if / @for:** nuevo control de flujo en plantillas. Más claro que *ngIf/*ngFor clásicos.
- **Lazy loading:** cargar características (código) sólo cuando se navega a ellas. Mejora el arranque.
- **SSR (Server-Side Rendering):** renderizar HTML en el servidor para entregar contenido rápido y amigable a buscadores.
- **Hydration:** al llegar al navegador, Angular conecta eventos sobre el HTML ya renderizado por SSR.
- **Detección de cambios (OnPush):** estrategia para renderizar menos y más rápido. No mutar objetos “en sitio”.
- **Zone.js / Zoneless:** mecanismo clásico para saber cuándo actualizar la UI (Zone.js). En “zoneless”, tú controlas con Signals y eventos.
- **Observable (RxJS):** flujo de valores en el tiempo (como un radio). Útil para peticiones, eventos, websockets.
- **NgRx/Store:** patrón para estado global complejo y trazable (acciones, reducers, selectors).
- **Build/Bundle:** proceso de empaquetar todo el código para producción (minificado, dividido en chunks).

Evita esto

Bootstrap (arranque) ≠ Bootstrap (CSS). *Aquí `bootstrapjs` significa iniciar la app.*

4 Tutorial guiado: de “Hola” a una mini app de tareas

Meta: pasar de un *hola* a una lista de tareas simple usando los conceptos clave, sin trucos.

4.1 Paso 1: Componente raíz simple

Crea un componente standalone con un título y un párrafo. Revisa el ejemplo de `HelloComponent` en Fundamentos.

4.2 Paso 2: Estado local con Signals

Agrega un contador con `signal(0)`, un botón que incrementa y otro que resetea. Ya tienes un dato que cambia y se refleja en la vista.

4.3 Paso 3: Lista de tareas (estado compuesto)

Define un tipo `Todo` y guarda una lista en `signal<ReadonlyArray<Todo>>([])`. Añade funciones `add(title)`, `toggle(id)` y `remove(id)` usando copias (no mutaciones).

Listing 7: Mini store local de Tareas con Signals

```

1 export type Todo = Readonly<{ id: string; title: string; done: boolean }>;
2
3 export class TodosComponent {
4   readonly todos = signal<ReadonlyArray<Todo>>([]);
5   readonly remaining = computed(() => this.todos().filter(t => !t.done).length)
  ;

```

```

6
7  add(title: string) {
8      const todo: Todo = { id: crypto.randomUUID(), title, done: false };
9      this.todos.update(list => [todo, ...list]);
10 }
11 toggle(id: string) {
12     this.todos.update(list => list.map(t => t.id === id ? { ...t, done: !t.done
13         } : t));
14 }
15 remove(id: string) {
16     this.todos.update(list => list.filter(t => t.id !== id));
17 }

```

Listing 8: Plantilla de tareas con @for y @if

```

1 <h2>Tareas (pendientes: {{ remaining() }})</h2>
2 <input #i placeholder="Nueva tarea"/>
3 <button (click)="add(i.value); i.value=''>Añadir</button>
4
5 @if (todos().length === 0) { <p>No hay tareas</p> } @else {
6     <ul>
7         @for (t of todos(); track t.id) {
8             <li>
9                 <label>
10                     <input type="checkbox" [checked]="t.done" (change)="toggle(t.id)"/>
11                     <span [class.done]="t.done">{{ t.title }}</span>
12                 </label>
13                 <button (click)="remove(t.id)"> </button>
14             </li>
15         }
16     </ul>
17 }

```

4.4 Paso 4: Separar lógica en un servicio

Mueve la gestión de tareas a un TodosService y inject() ese servicio en tu componente. Beneficio: testeable y reutilizable.

4.5 Paso 5: HTTP cuando haga falta

Sustituye los datos locales por una llamada con HttpClient. Usa switchMap para búsquedas y maneja errores con catchError. No recargues la vista manualmente: las señales u observables lo harán.

4.6 Paso 6: Routing para varias pantallas

Crea rutas simples: inicio (lista) y detalle (tarea). Usa loadComponent y, si necesitas datos previos, un ResolveFn.

Consejo Senior

Construye siempre en pasos pequeños: primero UI, luego estado local, después servicios, y finalmente HTTP y routing. Así entenderás cada pieza sin abrumarte.

5 Ecosistema Angular actual

Qué significa y por qué importa Angular moderno simplifica la creación de apps grandes: los componentes *standalone* eliminan la fricción de módulos, el nuevo control de flujo (`@if/@for`) hace las plantillas más legibles, y **Signals** ofrecen un modelo de estado simple y predecible. **@defer** y el *lazy loading* reducen el tiempo de carga inicial, mientras que SSR + *hydration* mejoran SEO y rendimiento percibido.

En la práctica Piensa en una app como páginas (rutas) compuestas por componentes. Cada componente tiene estado local (Signals), puede pedir datos (HttpClient) y reacciona a eventos de usuario. Divide tu app por **features** (dominios) y carga lo que hace falta cuando hace falta.

- Standalone por defecto: adiós a NgModule en apps nuevas; se configura con providers y bootstrapApplication.
- Nuevo control de flujo: @if, @for, @switch.
- Reactividad con **Signals**: signal, computed, effect.
- **@defer**: fragmentos de plantilla diferidos por condición, interacción u ociosidad.
- Builder esbuild y servidor rápido; SSR + *hydration* estable.
- Formularios tipados; takeUntilDestroyed para limpieza de suscripciones.

Evita esto

Evita mezclar demasiada lógica en plantillas. Mueve cálculos a `computed()` o a servicios, y reutiliza componentes para mantener la UI simple.

6 TypeScript para Angular (rápido y efectivo)

Qué aprenderás aquí Modelar datos y contratos claros para que el compilador te ayude: tipos de dominio, DTOs, utilitarios y técnicas de *narrowing* en eventos y HTTP.

Por qué type sobre interface type compone mejor (uniones/intersecciones). Usa interface cuando necesites *declaration merging* o describir clases públicas.

Narrowing efectivo Evita any. Usa typeof/in/instanceof para guiar al compilador cuando tratas con unknown o eventos del DOM.

Listing 9: Narrowing en handlers y datos HTTP

```
1 function onInput(e: Event) {
2   const el = e.target as HTMLInputElement; // narrowing seguro
3   const value = el.value.trim();
4 }
5
6 http.get<unknown>('/api/user').subscribe(x => {
7   if (isUser(x)) { setUser(x); } else { showError(); }
8 });
```

- Preferir type sobre interface salvo necesidad de declaración de fusión.
- Domina utilitarios: Partial, Required, Pick, Omit, Record, ReturnType.
- **Narrowing** con in, typeof, instanceof. Nunca uses any.

Consejo Senior

Crea tipos para IDs, DTOs y estado de UI. Te ahorrará bugs y te dará autocompletado útil.

7 Arquitectura y organización

Cómo organizar un proyecto Organiza por **features** (dominios), no por tipos de archivo. Cada feature contiene sus componentes, servicios y rutas. **shared/** para UI reutilizable y **core/** para servicios globales (auth, http). Con **standalone**, la mayoría de piezas no requieren NgModule.

7.1 Standalone y bootstrap

Arranque en 3 pasos 1) bootstrapApplication(AppComponent). 2) Registra providers globales (router, HttpClient, Material, etc.). 3) Añade interceptores/estrategias como funciones puras.

Listing 10: Arranque con standalone y providers raíz

```
1 // main.ts
2 import { bootstrapApplication } from '@angular/platform-browser';
3 import { provideHttpClient, withInterceptors } from '@angular/common/http';
4 import { provideRouter } from '@angular/router';
5 import { AppComponent } from './app/app.component';
6 import { routes } from './app/app.routes';
7
8 bootstrapApplication(AppComponent, {
9   providers: [
10     provideRouter(routes),
11     provideHttpClient(
12       withInterceptors([authInterceptor, loggingInterceptor])
13     ),
14   ]
15 });
```

7.2 Inyección de dependencias avanzada

Qué problema resuelve Compartir lógica (auth, cache, configuración) sin acoplar componentes a implementaciones concretas. Con inject() puedes obtener servicios también fuera de constructores (guards, interceptores, funciones).

Scopes útiles providedIn: 'root' = singleton global. providedIn: 'any' = una instancia por inyector (útil en *lazy*). providers: [...] en un componente = instancias por vista (evita fugas entre rutas).

Listing 11: Tokens, scopes y inject()

```
1 // tokens.ts
2 import { InjectionToken, inject, DestroyRef } from '@angular/core';
3
4 export type ApiConfig = Readonly<{ baseUrl: string; timeoutMs: number }>;
5 export const API_CONFIG = new InjectionToken<ApiConfig>('API_CONFIG');
6
7 export function useAbortable<T>(subscribe: (signal: AbortSignal) => Promise<T>)
8 {
9   const destroyRef = inject(DestroyRef);
10   const ac = new AbortController();
11   destroyRef.onDestroy(() => ac.abort());
12   return subscribe(ac.signal);
13 }
```

12 }

Evita esto

No acoples componentes a servicios concretos: usa `InjectionToken` para configuraciones y facilita los tests.

8 Componentes profesionales

Cómo pensar un componente Un componente debería: (1) recibir datos por `input()` y emitir acciones por `output()`, (2) gestionar sólo estado de UI local con Signals, (3) delegar I/O a servicios.

8.1 Inputs/Outputs, Signals y Change Detection

Qué hace cada pieza - **Inputs/Outputs**: API pública del componente. `input<number>(1)` tipa y da valor por defecto. `output<number>()` emite eventos tipados. - **Signals**: `signal` = estado, `computed` = derivado, `effect` = efectos secundarios (persistencia, analytics). - **OnPush**: evita renders innecesarios; cambia la vista cuando cambian inputs, signals o hay eventos.

Listing 12: Componente con Signals, OnPush y efecto de sincronización

```

1 import { Component, signal, computed, effect, input, output } from '@angular/
  core';
2
3 @Component({
4   selector: 'app-counter',
5   standalone: true,
6   templateUrl: './counter.html',
7   changeDetection: 0 // ChangeDetectionStrategy.OnPush
8 })
9 export class CounterComponent {
10   step = input<number>(1); // nuevo input con tipado fuerte
11   changed = output<number>(); // nuevo output funcional
12
13   count = signal(0);
14   doubled = computed(() => this.count() * 2);
15
16   constructor() {
17     effect(() => {
18       const c = this.count();
19       // sincroniza con almacenamiento, analytics, etc.
20       localStorage.setItem('count', String(c));
21     });
22   }
23
24   inc() {
25     this.count.update(c => c + this.step());
26     this.changed.emit(this.count());
27   }
28 }
```

Listing 13: Plantilla moderna con `@if/@for` y `@defer`

```

1 <div class="panel">
2   <h2>Contador: {{ count() }} (x2: {{ doubled() }})</h2>
3
4   @if (count() === 0) {
5     <p>Empieza a contar</p>
```

```

6   } @else {
7     <p>Llevas {{ count() }} clicks</p>
8   }
9
10  <button (click)="inc()">+{{ step() }}</button>
11
12  @defer (on idle; prefetch on viewport) {
13    <lazy-stats [value]="count()" />
14  } @placeholder {
15    <p>Cargando estadísticas.</p>
16  }
17 </div>

```

Evita esto

No mutar arrays/objetos en sitio. Usa update con copias ([...prev, nuevo]) para mantener OnPush eficiente.

8.2 Proyección, referencias y hooks

Cuándo usar content projection Cuando un componente *contenedor* (card, modal) debe renderizar contenido arbitrario del padre. Usa ng-content o @ContentChild para plantillas.

Listing 14: Content projection y limpieza reactiva

```

1  import { Component, ContentChild, TemplateRef } from '@angular/core';
2  import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
3  import { interval } from 'rxjs';
4
5  @Component({ selector: 'app-card', standalone: true, templateUrl: './card.html' })
6  export class CardComponent {
7    @ContentChild(TemplateRef) bodyTpl?: TemplateRef<unknown>;
8
9    ngOnInit() {
10      interval(1000)
11        .pipe(takeUntilDestroyed())
12        .subscribe(() => console.log('tick'));
13    }
14  }

```

9 Formularios tipados y validaciones

Elegir estrategia - **Reactive Forms**: más predecibles, testables y tipables. Preferidos en apps medianas/grandes. - **Template-driven**: válidos para formularios muy simples.

Mostrar errores de forma accesible Lee touched/dirty para decidir cuándo mostrar errores. Añade aria-invalid y aria-describedby.

9.1 Reactive Forms

Listing 15: FormGroup tipado y validadores personalizados

```

1  import { Component } from '@angular/core';
2  import { FormControl, FormGroup, Validators, NonNullableFormBuilder } from '@angular/forms';

```

```

3
4 interface ProfileForm {
5   name: FormControl<string>;
6   email: FormControl<string>;
7 }
8
9 @Component({ selector: 'app-profile', standalone: true, templateUrl: './profile
10   .html' })
11 export class ProfileComponent {
12   readonly form = new FormGroup<ProfileForm>({
13     name: new FormControl('', { nullable: true, validators: [Validators.
14       required] }),
15     email: new FormControl('', { nullable: true, validators: [Validators.
16       email] })
17   });
18 }

```

Listing 16: Validador asíncrono con cache y cancelación

```

1 import { AbstractControl, AsyncValidatorFn, ValidationErrors } from '@angular/
2   forms';
3
4 import { timer, map, switchMap, of } from 'rxjs';
5
6 export function emailTaken(check: (email: string) => Promise<boolean>):
7   AsyncValidatorFn {
8   return (ctrl: AbstractControl) => {
9     const email = ctrl.value as string;
10    if (!email) return of(null);
11    return timer(300).pipe( // debounce
12      switchMap(() => check(email)),
13      map(isTaken => (isTaken ? { emailTaken: true } as ValidationErrors : null
14        ))
15    );
16  };
17 }

```

10 Routing funcional, lazy y resolución de datos

Ideas clave Rutas como datos (Routes); loadComponent/loadChildren para *lazy*; Guards controlan acceso; Resolvers traen datos antes de entrar; usa data para metadatos y ActivatedRoute para leer params.

Listing 17: Definición de rutas standalone con guards funcionales

```

1 // app.routes.ts
2 import { Routes, provideRouter, CanActivateFn, ResolveFn } from '@angular/
3   router';
4 import { inject } from '@angular/core';
5 import { AuthService } from './auth.service';
6
7 const canActivateAuth: CanActivateFn = () => inject(AuthService).isLoggedIn();
8 const resolveUser: ResolveFn<User> = () => inject(AuthService).loadCurrentUser
9   ();
10
11 export const routes: Routes = [
12   { path: '', loadComponent: () => import('./home.component').then(m => m.
13     HomeComponent) },
14   { path: 'admin', canActivate: [canActivateAuth],
15     loadChildren: () => import('./admin/routes').then(m => m.ADMIN_ROUTES),

```

```

13   resolve: { user: resolveUser }
14 },
15 { path: '**', redirectTo: '' }
16 ];

```

Consejo Senior

Evita lógica de negocio en guards/resolvers. Delega en servicios y mantén funciones puras y testeables.

11 HTTP profesional: interceptores, errores, caché

Patrones esenciales - Interceptor de **auth**: añade token y reacciona a 401 (logout/refresh). - **Retry con backoff**: reintenta fallos transitorios sin saturar el servidor. - **Caché**: para GET idempotentes en listas/constantes.

Listing 18: Interceptor de auth y reintento exponencial

```

1 import { HttpInterceptorFn, HttpRequest, HttpHandlerFn } from '@angular/common/
  http';
2 import { inject } from '@angular/core';
3 import { catchError, retry, timer, mergeMap, throwError } from 'rxjs';
4
5 export const authInterceptor: HttpInterceptorFn = (req: HttpRequest<unknown>,
  next: HttpHandlerFn) => {
6   const token = inject(TokenService).get();
7   const cloned = token ? req.clone({ setHeaders: { Authorization: `Bearer ${
    token}` } }) : req;
8   return next(cloned).pipe(
9     retry({ count: 2, delay: (e, i) => timer(2 ** i * 200) }),
10    catchError(err => {
11      if (err.status === 401) inject(AuthService).logout();
12      return throwError(() => err);
13    })
14  );
15 };

```

Listing 19: Interceptor de caché simple por URL y método

```

1 const cache = new Map<string, unknown>();
2 export const cacheInterceptor: HttpInterceptorFn = (req, next) => {
3   if (req.method !== 'GET') return next(req);
4   const key = req.urlWithParams;
5   if (cache.has(key)) return of(new HttpResponse({ status: 200, body: cache.get(
    key) })).pipe(delay(0));
6   return next(req).pipe(tap((event) => {
7     if (event instanceof HttpResponse) cache.set(key, event.body);
8   }));
9 };

```

Evita esto

No caches peticiones mutantes (POST/PUT/DELETE) y recuerda invalidar caché tras cambios.

12 Estado: RxJS, Signals y NgRx

Cuándo usar cada uno - **Signals**: estado local de un componente o pequeñas vistas. - **RxJS**: flujos asíncronos (eventos, websockets, composición de peticiones) y *streams* compartidos. - **NgRx**: estado global complejo, múltiples actores, time-travel/debugging, reglas de negocio compartidas.

12.1 RxJS en componentes

Elegir operador correctamente `switchMap` cancela lo anterior (búsqueda), `concatMap` ordena (cola), `mergeMap` paraleliza (orden no importa), `exhaustMap` evita dobles envíos.

Listing 20: Elección correcta entre `mergeMap`/`switchMap`/`concatMap`/`exhaustMap`

```
1 // Regla mental:
2 // - switchMap: cancelar lo anterior (búsqueda incremental)
3 // - concatMap: en cola (operaciones ordenadas)
4 // - mergeMap: en paralelo (cuando el orden no importa)
5 // - exhaustMap: ignora hasta terminar (evita doble submit)
```

12.2 Signals para estado local

Regla práctica Deriva todo lo calculable con `computed` y mantén `signal` como fuente de verdad mínima.

Listing 21: Composición de estado con `signal`/`computed`/`effect`

```
1 export class TodosStore {
2   readonly todos = signal<ReadonlyArray<Todo>>([]);
3   readonly query = signal('');
4   readonly filtered = computed(() => this.todos().filter(t => t.title.includes(
5     this.query())));
6   add(title: string) { this.todos.update(arr => [...arr, { id: crypto.
7     randomUUID(), title, done: false }]); }
8 }
```

12.3 NgRx moderno

Estructura mental Acciones describen *qué pasó*, reducers calculan *nuevo estado*, selectors exponen *lecturas eficientes*. Efectos coordinan I/O.

Listing 22: Slice con `createFeature` y efectos tipados

```
1 import { createActionGroup, props, createFeature, createReducer, on,
2   createSelector } from '@ngrx/store';
3
4 export const UsersActions = createActionGroup({ source: 'Users', events: {
5   'Load': props<void>(),
6   'Loaded Success': props<{ users: User[] }>(),
7   'Loaded Failure': props<{ error: unknown }>(),
8 } });
9
10 export type UsersState = Readonly<{ users: User[]; loading: boolean; error?:
11   unknown }>;
12 const initialState: UsersState = { users: [], loading: false };
13
14 export const usersFeature = createFeature({
15   name: 'users',
16   reducer: createReducer(initialState,
```

```

15     on(UsersActions.load, (s) => ({ ...s, loading: true })),
16     on(UsersActions.loadedSuccess, (s, { users }) => ({ ...s, loading: false,
17       users })),
18     on(UsersActions.loadedFailure, (s, { error }) => ({ ...s, loading: false,
19       error })),
20   )
21   });
22   export const { name, reducer, selectUsers, selectLoading } = usersFeature;

```

13 Rendimiento y escalabilidad

Qué optimizar primero - Cambia a **OnPush**, usa **trackBy** y evita trabajo pesado en plantillas. - Divide rutas y usa **@defer** para cargar tarde lo no crítico. - Considera *zoneless* cuando controles el flujo con Signals y eventos propios.

- **OnPush** + inmutabilidad + **trackBy** en **@for**.
- Evita pipes pesados en plantilla; prefiera **computed** o pre-cálculo.
- Divide rutas y usa **@defer** para no bloquear TTI.
- Considera **zoneless**: `bootstrapApplication(App, providers: [provideZoneChangeDetection({ eventCoalescing: true, runCoalescing: true })] )` o `ngZone: 'noop'` y dispara CD manual con Signals.
- **SSR + Hydration**: mejora SEO y LCP.

Listing 23: Lista grande con **@for** y **trackBy**

```

1 // template
2 @for (item of items(); track item.id) {
3   <app-row [item]="item"/>
4 }

```

14 SSR, Hydration y despliegue

Cómo encajan SSR renderiza HTML en el servidor para entregar contenido rápido y rastreable. En cliente, *hydration* conecta eventos sin re-render completo. Protege el uso de APIs del navegador con `isPlatformBrowser`.

- Genera SSR: `ng add @nguniversal/express-engine`
- Verifica rutas *lazy* y uso de APIs del browser en `isPlatformBrowser`.
- Habilita hidratación con `provideClientHydration()` en el bootstrap.

Listing 24: Bootstrap con Hydration

```

1 import { provideClientHydration } from '@angular/platform-browser';
2 bootstrapApplication(AppComponent, { providers: [ provideClientHydration() ] })
3 ;

```

15 Testing eficaz

Estrategia mínima - Unit: componentes standalone con TestBed, servicios con dobles de prueba. - Integración: prueba flujo entre componentes/servicios. - E2E: escenarios críticos (login, compra). Usa *data-testid* y `retries`.

Listing 25: Prueba de componente standalone

```

1 import { TestBed } from '@angular/core/testing';
2 import { AppComponent } from './app.component';
3
4 describe('AppComponent', () => {
5   it('renderiza título', async () => {
6     await TestBed.configureTestingModule({ imports: [AppComponent] }).
7       compileComponents();
8     const fixture = TestBed.createComponent(AppComponent);
9     fixture.detectChanges();
10    expect(fixture.nativeElement.querySelector('h1').textContent).toContain('
11      Hola');
12  });
13 });

```

16 Accesibilidad, i18n y seguridad

Buenas prácticas inmediatas Añade roles/ARIA, gestiona el foco tras diálogos y navegación, evita [innerHTML] sin sanitizar, marca cadenas con *localizeparatraducción*.

- Usa roles/ARIA, focus management y cdkTrapFocus.
- i18n con @angular/localize; evita concatenaciones dinámicas.
- Sanear HTML: DomSanitizer; evita bypassSecurityTrustXxx salvo casos justificados.

17 Herramientas y calidad

Cómo empezar Activa ESLint con angular-eslint, configura Prettier y añade un pipeline de CI con build y tests. Analiza bundles antes de desplegar.

- ESLint con reglas angular-eslint; no-implicit-any, no-floating-promises.
- Prettier y formateo consistente.
- Analiza bundles: ng build -configuration production -stats-json + source-map-explorer.
- Monorepo: considera Nx para escalabilidad.

18 Patrones y anti-patrones

Patrón

Patrones: Presentational/Container, Facade para estado, Smart HTTP Service, Component Store para vistas complejas, Signals para estado local.

Evita esto

Anti-patrones: servicios god, mutaciones silenciosas, suscribir en servicios sin exponer Observables, abusar de any, lógica de negocio en plantillas.

19 Checklist de revisión senior

1. Tipado estricto activado; sin any/@ts-ignore.
2. OnPush y trackBy en listas.

3. División de rutas y @defer aplicado.
4. Interceptores para auth/errores; reintentos y backoff.
5. Limpieza con takeUntilDestroyed; cero fugas.
6. Accesibilidad validada y aria-* correcto.
7. Tests unitarios y de integración para piezas críticas.

20 Ruta de aprendizaje a experto (temario por niveles)

Esta ruta está pensada para construir bases sólidas, dominar el framework y entender sus internals. Muchas secciones enlazan con contenidos previos del documento.

20.1 1. Fundamentos del ecosistema Angular

- SPA: una única carga de documento, navegación cliente, estado en memoria, API vía HTTP/WS.
- Angular vs React vs Vue: Angular es un framework completo (router, DI, formularios, i18n); React y Vue son libraries que se amplían con ecosistema.
- TypeScript base: tipos, genéricos, union/intersection, narrowing, decoradores, módulos ES.
- Node/npm: scripts, semver, workspaces, npx.
- Modularidad y arquitectura: separación por **feature**, **core**, **shared**; patrones Presentational/Container.

20.2 2. Estructura básica de un proyecto

- CLI: crear, generar artefactos, ng generate, ng build, ng test.
- Carpetas: src/app, assets, environments. En apps modernas, **standalone** reemplaza NgModule.
- Componentes: @Component, plantilla, estilos, ciclo de vida.

20.3 3. Templates y data binding

- Interpolación, property/event binding, two-way con [(ngModel)] (formularios de plantilla).
- Directivas estructurales y control de flujo moderno: @if, @for, @switch.
- Directivas de atributo: [class], [style], [ngClass], [ngStyle].

20.4 4. Comunicación entre componentes

- @Input() y @Output() o APIs input()/output().
- ViewChild/ContentChild/ViewChildren.
- Servicios compartidos como canal de eventos/estado.

20.5 5. Servicios e inyección de dependencias

- @Injectable, providedIn, **scope** raíz vs componente.
- Jerarquía de inyectores, inject() y InjectionToken (ver Listado 9).

20.6 6. Ruteo y navegación

- Definición de Routes, navegación, parámetros, rutas hijas.
- Lazy loading con loadComponent/loadChildren. Guards y resolvers.

20.7 7. Formularios

- Template-driven y Reactive Forms; validaciones sincronas/asincronas; estados touched/dirty/pristine
- Mensajería de errores amigable y accesible.

20.8 8. Observables y RxJS

- Observables, suscripción, cancelación; operadores clave (map, filter, switchMap, catchError).
- Subject/BehaviorSubject/ReplaySubject; composición y shareReplay.

20.9 9. HTTP Client

- HttpClient: get/post/put/delete, cabeceras, parámetros, observe vs responseType.
- Interceptores, manejo de errores, JWT y refresh tokens.

20.10 10. Arquitectura avanzada

- División por dominio: feature folders, Core y Shared. Facades y servicios HTTP delgados.
- Smart vs Dumb Components; **Signals** para estado local, NgRx/Component Store para vistas complejas.

20.11 11. Gestión de estado

- NgRx: actions, reducers, selectors, effects. Alternativa **Signals** (Angular 16+), ver secciones previas.

20.12 12. Rendimiento y optimización

- OnPush, inmutabilidad, trackBy, @defer, zoneless, SSR + hydration.

20.13 13. Testing

- Unit con TestBed; mocks y espías; pruebas de servicios/pipes/directivas.
- E2E: **Cypress/Playwright** (Protractor está deprecado).

20.14 14. Despliegue y DevOps

- ng build --configuration production, entornos, variables, docker y CDNs.
- CI/CD con GitHub Actions (build, test, deploy).

20.15 15. Ecosistema avanzado

- Angular Material y CDK, Animaciones, Directivas/Pipes personalizadas, Angular Elements, WebSockets.

20.16 16. Buenas prácticas

- Convenciones, reutilización, arquitectura limpia, documentación, eslint y prettier.

21 TypeScript avanzado y tipado estricto en Angular

21.1 Tipos condicionales, utilitarios y “type-level programming”

Listing 26: Condicionales, infer y mapeados útiles

```
1 // Type guard para APIs estrictas
2 export function isUser(x: unknown): x is User {
3   return !!x && typeof x === 'object' && 'id' in (x as any) && 'email' in (x as
4     any);
5 }
6 // Condicional con infer: extraer payload de una Action NgRx
7 export type Action = { type: string; payload?: unknown };
8 export type PayloadOf<A extends Action> = A extends { payload: infer P } ? P :
9   never;
10 // Mapeados para construir formularios tipados desde un modelo
11 export type Formify<T> = { [K in keyof T]-?: FormControl<T[K]> };
12
13 type UserModel = { name: string; age: number };
14 export type UserForm = FormGroup<Formify<UserModel>>;
```

21.2 Decoradores y metadatos

- En Angular moderno, los decoradores se usan a nivel estático; el compilador genera metadatos internos de Ivy en tiempo de compilación (AOT/Ivy), por lo que no se requiere reflect-metadata en producción.
- Evita usar decoradores de clase personalizados en runtime; prefiere funciones puras y InjectionToken para configuración.

21.3 Chequeo de tipos en plantillas

- El Template Type Checker valida expresiones en HTML (strictTemplates). Actívalo siempre.
- Usa as const, narrowings y readonly para ayudar al compilador.

Listing 27: Tokens tipados y configuración segura

```
1 export interface FeatureFlags { readonly enableBeta: boolean; }
2 export const FEATURE_FLAGS = new InjectionToken<FeatureFlags>('FEATURE_FLAGS');
3
4 bootstrapApplication(AppComponent, { providers: [
5   { provide: FEATURE_FLAGS, useValue: { enableBeta: true } satisfies
6     FeatureFlags }
7 ]});
```

22 Angular por debajo del capó (internals esenciales)

22.1 Change Detection: de Zone.js a Signals

- **Zone.js**: parchea eventos/microtarear; tras eventos HTTP/DOM, marca vistas para comprobar. OnPush limita la propagación a inputs cambiados y eventos.
- **Zoneless + Signals**: los signals mantienen un grafo de dependencias; computed recalcula cuando cambian sus fuentes y las vistas leen señales con auto-dirty-check eficiente.
- APIs útiles: markDirty, ChangeDetectorRef.detectChanges(), runOutsideAngular para evitar trabajo extra.

22.2 Inyección de dependencias en Ivy

- Cada componente/directiva se asocia a vistas internas (LView/TView). La resolución de inject() sube por el árbol de inyectores hasta encontrar un proveedor o delega al inyector raíz.
- providedIn: 'any' crea un singleton por inyector; providers: [...] en componentes crea instancias por vista.

22.3 Compilación de plantillas (Ivy Instructions)

- La plantilla se transforma a instrucciones como instrucciones de creación/actualización separadas, optimizando el trabajo en cada ciclo de CD.
- El control de flujo @if/@for compila a bloques eficientes con block synthesis y dom diffing mínimo.

Listing 28: Esquema simplificado de plantilla compilada

```
1 // Pseudocódigo: create/update
2 function Template(rf: RenderFlags, ctx: Ctx) {
3   if (rf & 1) { /* create phase */
4     // elementStart('h1');
5     // text();
6     // elementEnd();
7   }
8   if (rf & 2) { /* update phase */
9     // textInterpolate(ctx.title);
10  }
11 }
```

22.4 Hydration y SSR

- SSR genera HTML inicial; en cliente, Angular hidrata: empareja nodos y adjunta listeners sin re-render completo.
- Evita usar APIs de navegador durante SSR; protege con isPlatformBrowser.

22.5 Builder y empaquetado

- El builder actual usa **esbuild** para velocidad; treeshaking, code-splitting, CSS inlining y optimización de imágenes.
- Habilita optimization, budgets y stats-json para auditar tamaño.

23 Plan de estudio recomendado (6 semanas)

1. Semana 1: Fundamentos TS, CLI, estructura y componentes. Labs: 3 componentes + comunicación básica.
2. Semana 2: Templates, control de flujo moderno, formularios template-driven y Reactive básicos.
3. Semana 3: HTTP, interceptores, RxJS esencial (switchMap, error handling), arquitectura por features.
4. Semana 4: Estado con Signals y NgRx intro; performance (OnPush, trackBy); routing avanzado y lazy.
5. Semana 5: SSR + hydration, zoneless, accesibilidad e i18n, animaciones y Material/CDK.
6. Semana 6: Testing unitario y E2E, seguridad (JWT/CSRF), CI/CD y despliegue; auditoría de bundles.

Consejo Senior

Estudia construyendo. Define un mini-producto (lista de tareas con auth + SSR) e incrementa complejidad cada semana aplicando los conceptos.

24 Material, CDK y diseño de UI

24.1 Angular Material (standalone)

Listing 29: Uso de Material en standalone

```
1 import { Component } from '@angular/core';
2 import { MatButtonModule } from '@angular/material/button';
3 import { MatIconModule } from '@angular/material/icon';
4
5 @Component({
6   selector: 'app-toolbar',
7   standalone: true,
8   imports: [MatButtonModule, MatIconModule],
9   template: `
10     <button mat-raised-button color="primary"><mat-icon>save</mat-icon> Guardar
11     </button>
12 `
13 })
14 export class ToolbarComponent {}
```

24.2 CDK Overlay y Portal

Listing 30: Overlay básico con CDK

```
1 import { Overlay, OverlayRef } from '@angular/cdk/overlay';
2 import { ComponentPortal } from '@angular/cdk/portal';
3
4 const overlayRef: OverlayRef = inject(Overlay).create();
5 overlayRef.attach(new ComponentPortal(MyPopupComponent));
```

25 Animaciones

Listing 31: Trigger con estados y transiciones

```
1 import { trigger, state, style, transition, animate } from '@angular/animations';
2
3 @Component({
4   // ...
5   animations: [
6     trigger('fade', [
7       state('in', style({ opacity: 1 })),
8       transition(':enter', [ style({ opacity: 0 }), animate('200ms ease-out')
9         ]),
10      transition(':leave', [ animate('150ms ease-in', style({ opacity: 0 }) ] )
11    ])
12  })
13 export class Panel {}
```

26 PWA y Service Worker

- Añade PWA: `ng add @angular/pwa`. Configura `ngsw-config.json` y `provideServiceWorker()`.

Listing 32: Bootstrap con Service Worker

```
1 import { provideServiceWorker } from '@angular/service-worker';
2 bootstrapApplication(AppComponent, {
3   providers: [ provideServiceWorker('ngsw-worker.js', { enabled: true }) ]
4 });
```

27 i18n y localización

Listing 33: Marcado con `$localize` y runtime i18n

```
1 const title = $localize`Hola, ${name}:bienvenido!`;
```

- Extrae mensajes: `ng extract-i18n` y compila con `@angular/localize`.

28 WebSockets y tiempo real

Listing 34: Servicio con `WebSocketSubject`

```
1 import { WebSocket } from 'rxjs/webSocket';
2 @Injectable({ providedIn: 'root' })
3 export class WsService {
4   private socket$ = WebSocket<string>('wss://example.com/ws');
5   messages$ = this.socket$.asObservable();
6   send(msg: string) { this.socket$.next(msg); }
7 }
```

29 Web Workers

Listing 35: Creación y comunicación con Worker

```
1 const worker = new Worker(new URL('./heavy.worker', import.meta.url));
2 worker.postMessage({ task: 'calc' });
3 worker.onmessage = (e) => console.log(e.data);
```

30 Seguridad en el navegador

- XSS: el binding Angular es seguro por defecto; [innerHTML] requiere sanitización. Evita `bypassSecurityTrustXxx` salvo casos controlados.
- CSP: usa nonces/hashes y evita `unsafe-inline`. Verifica 3rd-party scripts.

31 Estrategias de precarga y performance de rutas

Listing 36: PreloadAllModules y precarga personalizada

```
1 import { withPreloading, PreloadAllModules } from '@angular/router';
2 provideRouter(routes, withPreloading(PreloadAllModules));
```

Listing 37: Precarga condicional por data

```
1 export const routes: Routes = [
2   { path: 'admin', loadChildren: () => import('./admin/routes').then(m => m.
      ADMIN_ROUTES), data: { preload: true } }
3 ];
4
5 @Injectable({ providedIn: 'root' })
6 export class DataPreloadingStrategy implements PreloadingStrategy {
7   preload(route: Route, load: () => Observable<unknown>) {
8     return route.data?.['preload'] ? load() : of(null);
9   }
10 }
```

32 Monorepo y DX

- Nx: generators, executors, caché distribuida, grafo de dependencias.
- Auditoría: `ng build -stats-json + source-map-explorer`.